

Birding Coach — an overview

Forecasting when to go birding, not just where

Tharaka Jayalath

2026-08-12

Table of contents

Birding Coach — an overview	1
1. What it is	1
2. The one idea the whole thing rests on	2
3. Three repositories, and what each one is	2
4. How a bird sighting becomes a number on a phone	2
Why two Hugging Face datasets	3
5. The model, in plain words	3
What is currently fitted	4
Two evaluations, because there are two questions	4
6. What the app does	5
7. Decisions worth defending	5
8. Honest limits	6
9. What is not built	6
10. Running it	6
11. Data and citation	7

Birding Coach — an overview

Start here. This is the orientation document: what the thing is, why it is built the way it is, and where every piece lives. The two READMEs go deeper on their own halves — [this repository's](#) covers the model, and the app repository's covers the screens — but neither tells you what you are looking at from cold.

Written 2026-08-12, against the fit published that morning.

1. What it is

Every birding app answers *where*. Birding Coach answers *when*.

You pick a species, a hotspot, and a day in the coming week. It returns the probability that you would actually see that bird, hour by hour — the weather forecast applied to what birders have historically reported at that spot, at that time of day, in that week of the year.

The study region is **Allegheny County, Pennsylvania**. One county, on purpose; §8 explains why.

2. The one idea the whole thing rests on

Read this paragraph twice. Everything else is plumbing around it.

eBird lets a birder mark a checklist **complete** — a declaration that they listed every species they could identify. On a complete checklist, a species that does *not* appear is **evidence that the bird was not detected**. It is not missing data. It is a verified zero.

That is what makes a probability computable. A pile of sightings gives you a numerator and nothing else. Complete checklists give you the denominator: out of 412 complete lists submitted at Frick Park in week 31, how many contained a Wood Thrush? Now you can divide.

Every design decision downstream follows from protecting that zero:

- only complete checklists enter the fit — an incomplete one has no denominator
- the app's own "I listed everything" box is a claim recorded honestly, never a required checkbox (§7)
- a bird picked from photographs is flagged **uncertain**, because a wrong species also makes every absence around it wrong

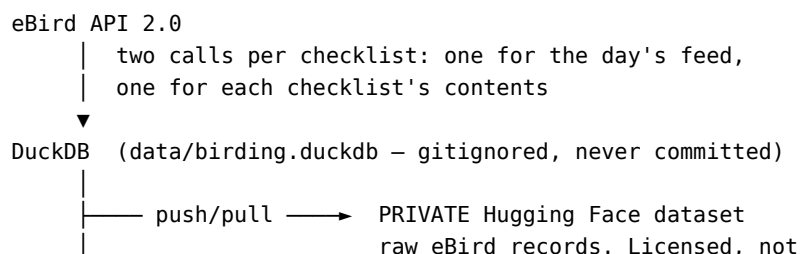
3. Three repositories, and what each one is

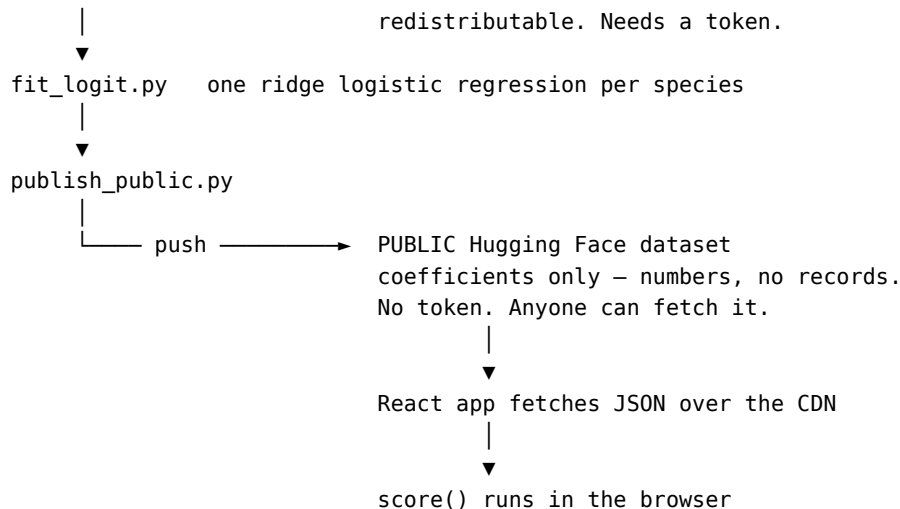
This is the part that confuses people first. There are three, and they are genuinely separate.

02-birding-coach	PUBLIC	github.com/taj771/birding-coach Python. Scrapes eBird, fits the model, publishes coefficients. You are here.
03-birding-coach-app	PRIVATE	github.com/taj771/birding-buddy-pro React + TanStack Start. What a user sees. Reads the published coefficients.
AI_certificate	LOCAL	No remote. Bootcamp coursework wrapper that the other two happen to sit inside.

The trap: the two project directories are nested inside the coursework directory on disk, but they are *not* part of its git repository. Running `git add 02-birding-coach` from the parent creates a gitlink — a broken pointer to a commit that lives somewhere else — and quietly detaches the project. Always work with `git -C 02-birding-coach ...` so the working directory cannot drift.

4. How a bird sighting becomes a number on a phone





Why two Hugging Face datasets

This split is the entire legal story of the project, so it is worth being precise about.

eBird records are free to use for non-commercial research and **must not be redistributed in raw form**. A fitted coefficient is derived work — a number describing a pattern, not a copy of anybody’s checklist — and publishing it is fine.

So the raw database goes somewhere private, and the coefficients go somewhere public. The consequence is the good part: **the browser needs no credentials at all**. No token in the front-end bundle, no proxy server, no auth layer in front of the model. The app just fetches a static JSON file.

The secondary consequence is that the pipeline is stateless. `data/` is gitignored, so a GitHub Actions runner starts with an empty disk every time. Each run pulls the shared database, appends what it scraped, and pushes it back. Both a laptop and CI write to it, so the pull **merges by natural key** (`sub_id`, `loc_id`, (`sub_id`, `species_code`)) rather than overwriting — running it twice adds nothing.

5. The model, in plain words

One ridge (L2-penalised) logistic regression per species. For a given species it estimates:

$P(\text{reported} \mid \text{hour, week, site, effort, weather})$

The terms, and what each is doing:

Block	Terms	What it absorbs
Effort	log duration, log distance, observer count, travelling	how hard you looked. The largest effect in the data.
Time of day	one term per hour, 06:00–20:00	dawn chorus, midday lull
Season	one term per week of the year	migration, breeding song, winter absence
Site	one term per hotspot	habitat, in one number

Block	Terms	What it absorbs
Weather	wind, temperature, precipitation, cloud	the thing actually under test

Effort is not optional, and this is the subtlest point in the model. Suppose it were left out. Rain makes birders cut a walk short, *and* rain makes birds quieter. Both push detections down. Omit duration and the rain coefficient silently absorbs both — you would publish “rain lowers your chances” when half of that is “rain shortened your walk”. Textbook omitted-variable bias. Conditioning on effort separates them.

Why ridge rather than ordinary logistic regression. There are 192 hotspots and some were visited four times. Unpenalised, the fit separates — a site where the bird was seen every one of its four visits gets an infinite coefficient, and roughly half the species were lost that way once site effects were added. Ridge shrinks small-sample sites toward the county average while letting a site with 89 visits keep its own estimate. Shrinkage follows the evidence, which is what an arbitrary “at least N visits” threshold was crudely approximating.

The cost is inference: ridge coefficients are biased on purpose, so **there are no valid p-values here**. That is acceptable because the app needs point predictions, not significance tests. Any claim about whether an effect is *real* would need a separate unpenalised fit.

What is currently fitted

8,129 complete checklists	101,553 observations
79 days	2025-01-29 .. 2026-08-05
2,062 sites scraped	192 with enough visits to model
73 species	18 weeks of the year covered

Two evaluations, because there are two questions

held out by SITE	median AUC 0.727	calibration MAE 0.030
held out by DATE	median AUC 0.801	calibration MAE 0.026

Never a random split — checklists from the same trip would leak across it.

The by-date number is what the app actually does: a hotspot the model knows, on a day it has not seen. It is the better of the two, and that gap is the evidence that site effects carry real information. Testing only by site would have hidden that completely, since a site the model has never seen cannot benefit from site effects by construction.

Calibration matters more than AUC here. AUC asks “does it rank correctly”; MAE of 0.026 says that when the app tells you 40%, the true rate is close to 40%. A ranking that is never honest about magnitude is not much use to someone deciding whether to get up at five.

6. What the app does

Four screens.

Forecast — pick a species and a hotspot, get the coming week hour by hour. Weather comes from Open-Meteo’s forecast at request time, which is why nothing has to be pushed daily: the seven-day window rolls forward on its own.

Log a sighting — a map, a time, effort fields, and a species list. What gets recorded is the outing, not just the bird: how long, how far, how many observers, and whether the birder was listing everything. Same shape as the eBird checklists the model is fitted on, deliberately.

Identification helper — the section a newcomer needs most. Someone saw a bird and cannot name it. The grid shows the 24 species most likely *at that spot, that hour, that week*, each with one photograph, ranked by the model.

It identifies nothing. It narrows 73 possibilities to 24 plausible ones and lets a person decide. Anything picked this way is stored with `uncertain = true`, because a guess and a confident identification must not look the same in the database. Photographs come from iNaturalist under Creative Commons licences, filtered at request time so an all-rights-reserved photo can never be displayed, with attribution under every image.

My logs — the birder’s own history.

Serving needs no Python at all. The fitted model is a lookup table plus a sum, so [serve/score.ts](#) is the entire scoring layer: look up the hour, the week and the site, add the effort and weather terms, apply the logistic transform.

7. Decisions worth defending

Short section, but this is the one a reviewer reads.

Scraping lags seven days. eBird submissions trail the outing — a weekend trip gets entered on Monday. Scraping fresh data captures only the people who log in the field, which is a biased and unrepresentative sample. Seven days is enough for submissions to settle.

Weeks are indexed by day-of-year, not by date. Week 31 is week 31 in any year, so last August’s checklists fill this August’s seasonal term. This is what makes a backfill of historical dates useful rather than merely interesting, and it is why the model can say something about a week it has not lived through yet.

numpy and scipy are pinned, though nothing imports them directly. They do the arithmetic that decides published coefficients. The same data was fitted on an arm64 laptop and an x86_64 GitHub runner and compared term by term: 17,526 terms, maximum absolute difference 7.9e-08, held-out metrics differing by at most 1.1e-09. That only held because both machines installed the same versions. To upgrade: bump the pins, refit, and diff against the previous run *before* publishing.

The complete-list checkbox does not block saving. It was built as a hard requirement, which sounds rigorous and is not. A gate on the save button does not produce complete checklists — it produces people ticking a box to get past it. That would corrupt the one field the entire method depends on. So the box is optional, the copy explains what ticking it means, and outings that decline it are stored and simply excluded at fit time. Record the truth about the record; filter later.

Only CC-licensed photographs, filtered in the request. An early version fetched the best-voted photo and then checked the licence; 24 of 104 came back all rights reserved. Filtering after the fact also meant falling back to nothing whenever the best photo was unusable. The licence filter now goes in the API call, with a null-licence rejection behind it as a second line.

Photos come from curated taxon photos, not observations. Ordering observation photos by votes sounds sensible until you notice the top result has two favourites out of thousands of photos — effectively random selection. Taxon photos are chosen by the iNaturalist community specifically to show what the species looks like.

8. Honest limits

State these before anyone else finds them.

- **Detection, not presence.** This predicts whether you would *report* a bird, not whether one is there. For a birder that is the right target. It is not the same quantity, and the distinction matters if anyone tries to read these numbers ecologically.
 - **One county.** Not a design preference — an arithmetic one. Building a denominator costs two API calls per checklist: about 4,650 for a month of one county, roughly 438,000 for a year of Pennsylvania, over a million for a month of the United States. Wider coverage means requesting the eBird Basic Dataset rather than scraping the API.
 - **18 of 52 weeks are fitted.** Ask about a week with no term and the model has nothing seasonal to say; the app reports that rather than guessing. The backfill exists to close this and has not finished.
 - **Effort is endogenous.** Birders extend a good outing, so duration is partly a *consequence* of success rather than only a cause. The effort coefficient is inflated by an unknown amount.
 - **Weather sits on a 25 km grid,** which attenuates its coefficients toward zero. The wind effect is probably understated.
 - **Observers self-select on weather.** Nobody birds in a storm, so the observed weather range is truncated and whoever does go out is unusual.
 - **The eBird feed truncates** any county-day above 200 checklists, under-sampling exactly the busiest spring mornings.
-

9. What is not built

- **The leaderboard.** Described in the brief, and `display_name` is already collected for it. Nothing renders it.
 - **Photo upload.** Deliberately deferred. Storing images is easy; reviewing them is not, and an unreviewed upload path is a moderation problem waiting.
 - **An unkept promise.** The sign-in page tells users their checklists improve the forecast. Nothing currently carries app-logged sightings from Supabase into the pipeline. Either build the path or change the copy — leaving both is the worst option.
-

10. Running it

Neither half is a repo-wide build; each project owns its own environment.

```
# pipeline
cd 02-birding-coach
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
cp .env.example .env          # EBIRD_API_KEY, HF_DATASET, HF_TOKEN, HF_PUBLIC_DATASET
.venv/bin/python pipeline/run.py daily      # scrape settled checklists    ~5 min
.venv/bin/python pipeline/run.py monthly    # refit and publish              ~10 min
```

`pipeline/run.py` is the **only** entry point; it shells out to the other scripts in a fixed order and every mode begins with `sync_db.py pull`. In CI, `.github/workflows/pipeline.yml` runs daily at 08:00 UTC and monthly on the 1st at 09:00 UTC, with a manual button that also offers **backfill** `<start>` `<end>` for filling calendar gaps.

The app repository has its own README covering `bun install`, the Supabase schema, and deployment through Lovable.

11. Data and citation

eBird API 2.0, Cornell Lab of Ornithology — free for non-commercial use, must be cited, **not redistributable in raw form**. This repository contains no eBird records.

Weather is ERA5 reanalysis and forecasts from [Open-Meteo](#).

Photographs are contributed to [iNaturalist](#) under Creative Commons licences and are attributed individually in the app.